

Git et GitHub : carnet de travaux pratiques

ENSUP Business School · Bachelor · 17 septembre 2026

Le projet : organiser la journée des associations

Vous préparez le dossier d'accueil d'une journée des associations ENSUP. Un nouvel étudiant doit y trouver où aller, à quelle heure et qui contacter. Le programme initial est incomplet : il manque un plan de repli, des indications d'accès et une vérification des horaires. Vous devrez aussi traiter une modification demandée en cours de journée.

Vous produisez des fichiers Markdown, lisibles dans GitHub. Aucun framework n'est nécessaire. Les compétences évaluées portent sur les versions, les branches, la collaboration et le diagnostic. Votre binôme relit vos propositions ; chacun conserve son historique personnel et rend son propre travail.

Le parcours : 8 heures de travail

1. Mise en route et première copie : 45 min (20 min d'explication, 25 min d'atelier).
2. Historique et index : 75 min (15 min de démo, 40 min de TP 1, 20 min d'enquête).
3. Branches et intégration : 60 min (15 min de démo, 30 min de TP 2, 15 min de comparaison).
4. Publication et collecte : 60 min (10 min de démo, 35 min de TP 3, 15 min de diagnostic).
5. Revue de code : 60 min (10 min de démo, 40 min de TP 4, 10 min de retour collectif).
6. Réparer et synchroniser : 75 min (10 min de démo, 45 min de TP 5, 20 min d'incident distant).
7. Livraison d'une version : 75 min (mission avec changement de consigne et preuves individuelles).
8. QCM et explication individuelle : 30 min (20 min de QCM, 10 min d'explications croisées).

Total : 480 minutes, hors pauses. Chaque durée comprend lecture, manipulation, contrôle et explication. Les activités d'enquête font partie de la journée ; ce ne sont pas des bonus facultatifs.

Les trois adresses à utiliser

Dépôt de classe : <https://github.com/AbidHamza/ensup-git-github-2026>

QCM : <https://abid-hamza.com/ensup-git/qcm/>

Collecte : <https://github.com/AbidHamza/ensup-git-github-2026/pulls>

Les rendus GitHub sont publics. Utilisez votre identifiant GitHub et votre adresse noreply exacte (Settings > Emails), sans ajouter téléphone, adresse personnelle ou mot de passe. Le QCM conserve vos résultats dans l'espace formateur, pas dans le dépôt public.

Comment lire les commandes

Ouvrez **Git Bash**, pas PowerShell. Tapez une commande à la fois. `pwd` indique le dossier, `ls` liste son contenu, `cd ..` remonte d'un niveau, `q` quitte le journal paginé. Enregistrez vos fichiers dans l'éditeur avant de consulter Git. Vérifiez les extensions Windows : README.md, pas README.md.txt.

Dans les commandes, remplacez IDENTIFIANT par votre identifiant GitHub, exactement écrit, sans espace. Les textes en MAJUSCULES sont à remplacer. Ne recopiez pas le symbole \$ des captures. Un message d'erreur est un résultat à lire : ne passez pas à la suite tant que le contrôle demandé n'est pas obtenu.

Le cours, la documentation et le mémo sont autorisés pendant les TP. À chaque contrôle, l'étudiant au clavier explique son action au binôme, qui vérifie le résultat. Chacun effectue les manipulations sur sa propre copie.

Atelier 0 : préparer sa copie · 25 minutes

A. Fork : votre copie sur GitHub · 7 minutes

Connectez-vous à GitHub. Ouvrez le dépôt de classe, cliquez **Fork**, choisissez votre compte comme propriétaire et conservez le nom `ensup-git-github-2026`. Cliquez **Create fork**. Vérifiez que le haut de page affiche **votre compte**, avec la mention de provenance du dépôt AbidHamza.

Cette copie vous donne le droit de pousser chez vous. Vous proposerez ensuite vos commits au formateur par pull request, sans avoir besoin d'un accès en écriture à son dépôt.

B. Clone : votre copie sur le PC · 8 minutes

Ouvrez Git Bash dans votre dossier de travail (hors de tout autre dépôt).

```
git --version
git clone https://github.com/IDENTIFIANT/ensup-git-github-2026.git
cd ensup-git-github-2026
git config user.name "IDENTIFIANT"
git config user.email "VOTRE_ADRESSE_NOREPLY_EXACTE"
git remote add upstream https://github.com/AbidHamza/ensup-git-github-2026.git
git remote -v
git status
```

Attendu : main est la branche courante ; origin pointe vers votre compte et upstream vers AbidHamza. La configuration de l'auteur n'est pas une connexion GitHub. Le clone a déjà un historique : ne lancez pas `git init` ici.

C. Votre espace de travail · 10 minutes

```
mkdir -p rendus/IDENTIFIANT
cd rendus/IDENTIFIANT
```

Gardez ce dossier courant pour les TP, sauf indication explicite. Dans l'éditeur, créez votre README.md dans ce dossier avec :

```
# Journée des associations ENSUP
Un guide pour préparer sa première visite sur le campus.
```

Créez aussi `journal.md` : titre « Carnet de bord », identifiant GitHub, puis une rubrique « Mes preuves ». Vous complèterez ce carnet après chaque atelier avec les commandes observées et vos explications. N'y mettez pas

les réponses au QCM.

Contrôle : `pwd` se termine par votre identifiant ; `git status --short` signale les deux nouveaux fichiers ; `git log -1 --oneline` montre le commit de départ du formateur.

Dépannage : dépôt introuvable : relisez l'URL de votre fork ; dossier déjà présent : ne clonez pas dedans, ouvrez la copie existante ; git introuvable : demandez l'installation au formateur. Si GitHub est inaccessible, le formateur fournit la copie de secours et reprend l'envoi avec vous ensuite.

TP 1 : Construire un historique utile

40 minutes · Individuel puis vérification avec un voisin.

Départ : atelier 0 terminé, dans `rendus/IDENTIFIANT`. **Arrivée** : au moins trois commits utiles, un fichier ignoré et un dépôt propre après avoir commité le carnet.

Étape A : Le premier commit · 7 minutes

1. Exécutez `git status` et expliquez l'état du fichier.
2. Préparez seulement `README.md`.
3. Relisez ce que vous préparez.
4. Enregistrez un premier commit.

```
git add README.md
git diff --cached
git commit -m "docs: presenter la journée ENSUP"
git log --oneline
git status
```

À écrire : que fait `add` que ne fait pas l'enregistrement dans l'éditeur ?

Réponse : _____

Étape B : Le piège de l'index · 12 minutes

1. Ajoutez `Entrée gratuite.` à la fin de `README.md`. Enregistrez.
2. Exécutez `git diff`, puis `git add README.md`.
3. Ajoutez maintenant `Inscription sur place.` au fichier. Enregistrez **sans refaire add**.
4. Exécutez les trois commandes suivantes :

```
git status --short
git diff --cached
git diff
```

Prédiction avant de continuer : si vous committez maintenant, quelle phrase manque dans le commit ? Pourquoi ?

Réponse : _____

5. Refaites `git add README.md` pour inclure les deux phrases, puis créez le deuxième commit :

```
git commit -m "docs: preciser les conditions d acces"  
git show HEAD
```

Attendu : show contient les deux phrases ajoutées. Les identifiants de vos commits sont propres à votre dépôt ; il ne faut pas reproduire ceux des slides.

Étape C : Choisir ce qui reste local · 9 minutes

Créez .gitignore avec :

```
.env  
notes-privees.txt
```

Créez .env avec **uniquement** DEMO=FAUSSE_VALEUR. N'utilisez aucun vrai mot de passe. Créez aussi notes-privees.txt contenant une note de test.

```
git status --short  
git check-ignore -v .env  
git add .gitignore  
git commit -m "chore: ignorer les fichiers locaux"
```

Attendu : .gitignore est versionné. .env et notes-privees.txt ne figurent pas parmi les fichiers à commiter. .gitignore n'enlève pas de l'historique un fichier déjà suivi.

Avant la vérification, complétez journal.md avec l'explication de l'index, puis `git add journal.md` et `git commit -m "docs: expliquer le fonctionnement de l index"`. Ce commit de preuve s'ajoute aux trois commits du projet.

Étape D : Vérification croisée · 7 minutes

Montrez à votre voisin :

```
git log --oneline  
git status  
git ls-files
```

Il vérifie trois commits avec des messages utiles, un dépôt propre après avoir commité le carnet et l'absence de .env dans ls-files. Expliquez ensemble la différence entre fichier enregistré, index et commit.

Trace individuelle · 5 minutes

Conservez dans votre fiche de rendu un court extrait du log et votre explication de l'index. Ne placez pas les captures dans le dépôt du projet si cela n'est pas demandé.

Aides graduées : 1. Commencez par status. 2. Comparez diff et diff --cached. 3. Demandez au formateur de vous aider à localiser le problème, sans lui déléguer toute la manipulation.

Bonus : modifiez deux paragraphes, puis explorez `git add -p`. Expliquez pourquoi préparer seulement une partie peut aider à créer des commits cohérents.

Enquête 1 : le fichier disparu du diff · 20 minutes

Situation : vous créez acces.md dans votre dossier de rendu. Son contenu : « Entrée visiteurs : porte principale. Présenter son invitation à l'accueil. » Votre voisin affirme que `git diff` doit montrer ce texte.

1. Pendant 3 min, écrivez votre prédiction avant d'exécuter `git diff`.
2. Pendant 5 min, comparez `git status --short`, `git diff`, puis `git add acces.md` et `git diff --cached`.
3. Pendant 5 min, expliquez pourquoi un fichier non suivi n'apparaît pas dans le diff ordinaire. Commitez le fichier.
4. Pendant 7 min, retrouvez le commit d'ajout avec `git log --oneline -- acces.md`, puis `git show IDENTIFIANT_DU_COMMIT`. Ajoutez la preuve et votre explication au carnet, commitez-le.

Réussite : le binôme sait retrouver une modification à partir du nom du fichier, sans parcourir toute l'histoire à la main.

TP 2 : Une branche pour une évolution

30 minutes · Travail individuel, prédictions en binôme.

Départ : TP 1 terminé sur main, dépôt propre. **Objectif :** ajouter le programme via une branche.

1. Isoler le travail · 5 minutes

```
git status
git switch -c feature/programme
git branch
```

Créez `programme.md` :

```
# Programme
09h00 : accueil : Hall A
10h00 : découverte des associations : Salle 1
11h00 : atelier découverte : Salle 2
```

2. Enregistrer l'évolution · 8 minutes

```
git add programme.md
git diff --cached
git commit -m "feat: ajouter le programme"
```

Avant de revenir sur main, prédisez quels fichiers seront visibles.

3. Observer puis intégrer · 10 minutes

```
git switch main
ls
git merge feature/programme
git log --graph --oneline --all
git status
```

Attendu : avant la fusion, `programme.md` n'existe pas sur main. Après, il est présent. Une fusion **fast-forward** est normale si main n'a pas avancé depuis la création de la branche. Dans ce cas, aucun nouveau commit de fusion n'est nécessaire.

4. Expliquer · 7 minutes

Votre binôme doit pouvoir répondre : sur quelle branche faut-il être pour intégrer une évolution **dans main** ? Quelle commande prouve que le contenu est présent ? Dessinez le graphe avant et après.

Aide : `git branch` marque la branche courante avec *. Si `switch` refuse, ne supprimez rien : inspectez `status` et enregistrez le travail utile.

Bonus : créez deux branches depuis le même point, modifiez des fichiers différents puis fusionnez-les. Observez que divergence ne signifie pas nécessairement conflit.

Enquête 2 : divergence sans conflit · 15 minutes

Depuis `main` propre, créez `feature/accessibilite`, créez `accessibilite.md` avec « Accès de plain-pied : entrée principale. », puis `add` et `commit`. Revenez sur `main`, ajoutez « Accueil des visiteurs à partir de 09h00. » à `README.md`, puis `add` et `commit`. Les deux branches ont maintenant avancé sur des fichiers différents.

```
git log --graph --oneline --all
git merge --no-ff feature/accessibilite -m "merge: ajouter les indications d accessibilite"
git show --no-patch --format="%h %p %s" HEAD
git status
```

Répartissez le temps : 6 min de création, 4 min de fusion, 5 min de dessin et d'explication. Comparez avec le `fast-forward` du TP 2. Cette fois le `commit` de fusion a deux parents ; la divergence n'a pas provoqué de conflit car les fichiers modifiés sont distincts. Notez les deux identifiants de parents dans le carnet et commitez-le.

TP 3 : envoyer au formateur · 35 minutes

A. Publier vos commits dans votre fork · 10 minutes

Revenez sur `main` et terminez les modifications du carnet. `git status` doit annoncer un arbre propre. Vérifiez `origin` : il doit désigner votre propre compte.

```
git switch main
git remote -v
git log -3 --oneline
git push -u origin main
```

Suivez la connexion proposée par le navigateur ou le gestionnaire d'identifiants. Le mot de passe du site ne remplace pas l'authentification Git HTTPS. Ne collez jamais de token dans une commande à projeter.

Attendu : votre dossier apparaît dans le fork sur GitHub, avec le dernier `commit` local. Rien n'est encore fusionné dans le dépôt de classe.

B. Ouvrir la PR de collecte · 12 minutes

Dans votre fork, utilisez **Contribute > Open pull request**. Si ce bouton n'apparaît pas, ouvrez **Pull requests > New pull request**, puis **compare across forks**.

Vérifiez les quatre champs : **base repository** = **AbidHamza/ensup-git-github-2026**, **base** = **main**, **head repository** = **votre fork**, **compare** = **main**. Il est normal que les deux branches s'appellent `main` : elles appartiennent à deux dépôts différents.

Titre : Rendu IDENTIFIANT : journée des associations. Dans la description, indiquez votre dossier, les ateliers terminés, le lien vers votre journal et un point à faire relire. Ouvrez la PR. Elle reste ouverte pendant les ateliers ; seul le formateur décide de son intégration finale.

Contrôle avant envoi : Files changed ne contient que votre dossier rendus/IDENTIFIANT/. N'envoyez pas de modification du README de classe, des supports ou du dossier d'un autre étudiant.

C. Constater la mise à jour automatique · 8 minutes

Ajoutez dans journal.md l'URL de cette PR et une explication de la différence entre fork, clone et branche. Commitez puis `git push origin main`. Actualisez l'onglet **Commits** de la PR : le nouveau commit y apparaît sans créer une deuxième PR de collecte.

D. Vérification croisée · 5 minutes

Ouvrez la PR de votre voisin depuis le dépôt **AbidHamza**. Vérifiez son auteur, son dossier et son dernier commit. Chaque étudiant doit pouvoir montrer ces trois preuves.

Enquête 3 : pourquoi mon push est refusé ? · 15 minutes

Sur papier, associez chaque message à un diagnostic et à une vérification, puis comparez en binôme. Il n'est pas demandé de provoquer un refus d'accès.

1. `Permission denied` : votre origin vise-t-il votre fork ou le dépôt du formateur ? Quel compte est connecté ?
2. `non-fast-forward` : des commits ont-ils été créés sur GitHub depuis votre dernier envoi ? Quelle commande récupère les repères sans fusionner ?
3. `nothing to commit` alors que le fichier paraît modifié : est-il enregistré, dans le bon dossier, ignoré ou déjà commité ?

Répartition : 5 min de diagnostic individuel, 5 min de discussion, 5 min de mise en commun. Écrivez une commande de contrôle par cas dans votre carnet, puis commitez et poussez. Un push forcé n'est pas une réponse attendue.

TP 4 : faire relire une modification · 40 minutes

Chaque étudiant travaille dans **son fork**, tandis que son binôme le relit. Les deux font les étapes ci-dessous, puis inversent auteur et reviewer. Le reviewer n'a pas besoin d'être collaborateur pour commenter une PR publique ; l'auteur fusionne dans son propre fork.

A. Préparer une petite évolution · 10 minutes

Depuis votre dossier de rendu, sur main propre :

```
git switch -c feature/equipe
```

Créez `equipe.md` avec deux fonctions : « Accueil visiteurs » et « Coordination des salles ». Attribuez-les à des personnes fictives. Ajoutez aussi une consigne pour joindre l'accueil **sur place**, sans coordonnées réelles. Commitez le fichier puis poussez :

```
git add equipe.md
```

```
git commit -m "docs: repartir les roles de l accueil"  
git push -u origin feature/equipe
```

B. Ouvrir une PR interne à votre fork · 7 minutes

Dans GitHub, **Pull requests > New pull request**. Attention : **base repository et head repository doivent tous deux être VOTRE fork**, base main, compare feature/equipe. Vérifiez ces champs même si GitHub propose le dépôt du formateur par défaut.

Décrivez l'objectif, les fichiers modifiés et le contrôle effectué. Copiez l'URL pour votre binôme. Cette PR de travail est distincte de la PR de collecte restée ouverte chez AbidHamza.

C. Relire et corriger · 13 minutes

Le binôme ouvre Files changed et lit chaque ligne. Il rédige un commentaire exploitable : par exemple « Qui prend le relais si la coordination des salles est absente ? Ajoute une consigne précise. » Évitez « bien » ou « à améliorer » seuls. S'il démarre une revue, il termine par **Review changes > Comment > Submit review**.

L'auteur complète equipe.md dans la **même branche feature/equipe**. Il commite la correction et refait git push. Il répond à la remarque avec ce qui a changé. Le reviewer vérifie le nouveau diff et laisse un commentaire de validation. Reproduisez cette revue dans l'autre sens sur la PR de votre binôme.

D. Fusionner dans votre fork et actualiser · 10 minutes

Le propriétaire du fork choisit **Create a merge commit** dans le menu du bouton de fusion et confirme. Conservez les commits individuels : n'utilisez pas Squash and merge pour ce rendu.

```
git switch main  
git pull --ff-only origin main  
git log -5 --oneline
```

Le fichier equipe.md est présent sur main local. La PR de collecte se met à jour puisque votre main distant a avancé. Ajoutez au journal les deux URLs (PR écrite et PR relue), le commentaire utile et le commit de correction ; commitez et poussez.

Retour collectif : une revue exploitable · 10 minutes

Chaque binôme choisit une remarque précise et montre le diff avant/après. Le formateur fait comparer une remarque sur le contenu à une préférence de présentation. Réussite : la correction répond réellement au problème soulevé et le lien avec le commit est visible.

TP 5 : Le laboratoire des erreurs

45 minutes · Individuel, puis explication croisée.

Le laboratoire est un sous-dossier de votre rendu. Ses commits seront donc également visibles dans votre PR de collecte. Revenez sur main avec un état propre avant de commencer. Les commandes de création de fichiers ci-dessous fonctionnent dans **Git Bash**. > remplace le contenu : ne les exécutez que dans le laboratoire.

A. Préparer le laboratoire · 4 minutes

Depuis votre dossier rendus/IDENTIFIANT, dans le même dépôt que les TP précédents :

```
git switch main
mkdir labo-conflit
cd labo-conflit
printf 'Accueil : 09h00\n' > horaire.txt
git add horaire.txt
git commit -m "init: horaire de reference"
```

B. Désindexer sans perdre · 5 minutes

```
printf 'Note provisoire\n' > brouillon.txt
git add brouillon.txt
git restore --staged brouillon.txt
git status --short
```

Attendu : brouillon.txt existe toujours et redevient non suivi. Pour la suite, préparez-le et commitez-le :

```
git add brouillon.txt
git commit -m "test: ajouter un brouillon"
```

C. Annuler le dernier commit · 5 minutes

```
git revert --no-edit HEAD
git log -3 --oneline
git status
```

Attendu : le fichier brouillon disparaît, le commit d'ajout reste dans l'historique et un nouveau commit inverse apparaît. Vous n'avez pas effacé l'histoire.

D. Créer deux propositions incompatibles · 8 minutes

```
git switch -c feature/horaire
printf 'Accueil : 10h00\n' > horaire.txt
git add horaire.txt
git commit -m "feat: proposer 10h00"
git switch main
printf 'Accueil : 09h30\n' > horaire.txt
git add horaire.txt
git commit -m "docs: annoncer 09h30"
git merge feature/horaire
```

Attendu : Git signale un conflit. C'est volontaire. git status indique un fichier non fusionné. Dans l'éditeur, horaire.txt contient les deux propositions et les marqueurs <<<<<<, =====, >>>>>>.

E. Décider et résoudre · 8 minutes

Le commanditaire valide **09h45**. Dans l'éditeur, remplacez tout le contenu par la ligne suivante et enregistrez :

Accueil : 09h45

Puis :

```
git add horaire.txt
git commit -m "fix: convenir de 09h45"
git status
git log --graph --oneline --all
```

Attendu : dépôt propre, aucun marqueur restant, horaire validé. Le commit de fusion relie les deux histoires.

F. Expliquer · 5 minutes

Montrez le résultat à votre binôme. Répondez : pourquoi Git n'a-t-il pas choisi seul ? Quelle version était HEAD ? À quoi sert `git merge --abort` pendant une fusion en cours ?

Bonus guidé : sur une nouvelle modification locale jetable d'un fichier déjà suivi, comparez `git diff`, puis `git restore NOM_DU_FICHIER`. Cette dernière commande remplace le contenu de travail par celui de l'index par défaut : ne la lancez pas sur un travail à conserver.

G. Conserver les preuves et envoyer · 10 minutes

Avant de quitter le laboratoire, lancez `git log --graph --oneline --all` et `git show --no-patch --format="%h %p %s" HEAD`. Recopiez dans votre carnet les deux parents du commit de fusion. Puis revenez dans votre dossier de rendu avec `cd ...`. Complétez `journal.md` : erreur de départ, deux propositions, décision retenue, commandes de fin de fusion. Commitez le carnet puis `git push origin main`. Vérifiez que la PR de collecte contient aussi ces commits.

Contrôle oral : votre voisin désigne un commit du graphe ; expliquez son rôle sans relire les commandes du livret.

Incident distant : le programme modifié en ligne · 20 minutes

Restez dans votre dossier de rendu, sur main propre. Votre PR de collecte est toujours ouverte.

1. **5 min** : sur GitHub, dans votre fork et sur main, ouvrez votre `programme.md`, cliquez sur le crayon. Ajoutez « 12h00 : visite libre des stands, Hall A. » et commitez directement sur main. Si GitHub propose une branche, choisissez explicitement main. Ne lancez pas encore pull sur le PC.

2. **5 min** : localement, ajoutez « Prévoir une arrivée dix minutes avant le premier atelier. » à `README.md` et commitez. Un `git push origin main` est maintenant normalement refusé : les deux histoires ont avancé.

3. **5 min** : récupérez et observez avant d'intégrer :

```
git fetch origin
git log --graph --oneline --all -10
git merge origin/main -m "merge: recuperer le programme publie en ligne"
git push origin main
```

4. **5 min** : vérifiez la présence des deux changements sur GitHub. Écrivez dans `journal.md` pourquoi fetch seul n'aurait pas ajouté la visite libre à votre fichier local. Commitez et poussez cette explication.

Attendu : fusion sans conflit car les modifications portent sur des fichiers différents. Si une autre erreur apparaît, relevez le message exact avec le formateur avant de continuer. Ne forcez pas le push.

Mission finale : une version utilisable · 75 minutes

Le secrétariat vous demande une version prête à remettre aux visiteurs. Vous reprenez votre propre dossier. Votre binôme est votre relecteur ; chacun doit produire une nouvelle PR de travail dans son fork, différente de celle du TP 4.

Cahier des charges

1. README.md : objectif, consignes d'arrivée et liens Markdown fonctionnels vers programme, accès et équipe.
2. programme.md : au moins cinq créneaux ordonnés, une activité et un lieu pour chaque créneau, aucun chevauchement pour un même parcours visiteur.
3. acces.md : point de rendez-vous, chemin vers l'accueil, solution si l'entrée habituelle est fermée.
4. equipe.md : fonctions, relais en cas d'absence, consigne de contact sur place.
5. journal.md : preuves des ateliers, URLs de vos PR, revue effectuée, problème rencontré, diagnostic et solution.

Étape 1 : découper la demande · 10 minutes

Dans votre fork, créez une issue (Issues > New issue) nommée « Préparer la version visiteurs ». Activez Issues dans Settings > General > Features si nécessaire. Écrivez trois critères vérifiables tirés du cahier des charges. Depuis main actualisé et propre, créez feature/livraison. Notez le numéro de l'issue.

Étape 2 : réaliser et vérifier · 20 minutes

Modifiez les documents. Faites au moins deux commits cohérents : un pour le parcours et les horaires, un pour les consignes et les liens. Avant chaque commit, relisez `git diff --cached`. Ouvrez les fichiers sur GitHub après push et testez chaque lien du README.

Étape 3 : demander une revue · 15 minutes

Ouvrez une PR **dans votre fork**, de feature/livraison vers main. Mentionnez Closes #NUMERO avec le numéro de votre issue. Le binôme teste votre parcours : peut-il trouver où se présenter, quoi faire si l'entrée est fermée et à qui demander de l'aide ? Il commente au moins une ligne. Faites de même pour sa PR.

Étape 4 : traiter le changement du commanditaire · 15 minutes

À la minute 45, le formateur annonce un changement. Notez-le dans la PR, adaptez tous les fichiers concernés et ajoutez un commit de correction sur la même branche. Demandez au binôme une seconde lecture. Il vérifie notamment que README et programme ne se contredisent plus.

Étape 5 : livrer et rendre · 15 minutes

Fusionnez dans votre fork avec Create a merge commit. Revenez sur main et `git pull --ff-only origin main`. Complétez le journal, committez et poussez. Créez un tag annoté pour retrouver la livraison :

```
git tag -a v1.0 -m "Version visiteurs relue"
git push origin v1.0
git status
```

Ouvrez la PR de collecte sur le dépôt du formateur. Son onglet Commits doit montrer tout votre travail, y compris les corrections, le laboratoire et la fusion finale. Complétez la description de la PR avec les liens vers vos deux PR de travail, la revue du binôme et le tag de votre fork. **Ne fusionnez pas cette PR : le formateur la relit.** Le tag reste dans votre fork ; le lien permet de le retrouver.

Contrôle de rendu individuel

Votre dossier doit contenir README.md, programme.md, acces.md, accessibilite.md, equipe.md, journal.md, .gitignore et labo-conflit/horaire.txt. Le fichier .env est uniquement un exemple local ignoré. La PR doit permettre de retrouver au minimum trois commits de contenu, deux corrections après revue, un revert et un commit de résolution du conflit. Ne réécrivez pas les anciens commits pour gonfler artificiellement le nombre.

Dans le carnet, répondez en quelques phrases : que copie add ? Comment prouver qu'un commit a été poussé ? Pourquoi Git n'a-t-il pas choisi l'horaire ? Que reste-t-il après revert ? Montrez un diff dont vous êtes l'auteur.

QCM et explication · 30 minutes

Ouvrez <https://abid-hamza.com/ensup-git/qcm/> et saisissez votre identifiant GitHub pour que le formateur puisse rapprocher copie et rendu. Répondez individuellement aux 20 questions pendant 20 minutes, sans mémo. Validez la copie et conservez son numéro de réception. En cas de réseau indisponible, utilisez QUIZ-ETUDIANT.pdf et remettez la copie au formateur.

Pendant les 10 dernières minutes, chacun explique à son binôme un diff et un incident. Le formateur vérifie les preuves individuellement au fil des TP et complète les contrôles manquants. Le QCM ne remplace pas la réalisation des manipulations.

Pour ceux qui terminent plus tôt

Après validation des preuves obligatoires : préparez une petite modification de README, utilisez `git stash push -m "consigne en cours"`, observez status, puis `git stash pop` ; committez le résultat utile. Comparez `git show v1.0:README.md` depuis la racine avec `git show v1.0:rendus/IDENTIFIANT/README.md` : le premier désigne le README de classe. Expliquez l'importance du chemin. Ces exercices occupent 15 à 20 minutes sans ajouter de notions à l'évaluation.

Références utiles

Commandes : <https://git-scm.com/docs>

Fork et clone : <https://docs.github.com/en/pull-requests/how-tos/work-with-forks/fork-a-repo>

Revue : <https://docs.github.com/en/pull-requests>